1. Büyük Dil Modellerine Teknik Bakış

Transformer mimarisi, eğitim, inference, RAG ve production serving üzerine teknik eğitim dokümanı

Büyük dil modelleri (Large Language Models - LLM), doğal dildeki örüntüleri çok büyük metin koleksiyonlarından öğrenen ve bir sonraki tokenı tahmin etme problemi üzerinden eğitilen sinir ağı modelleridir. Güncel LLM mimarilerinin büyük bölümü Transformer ailesine dayanır. Model, bir metni kelime olarak değil token adı verilen daha küçük birimlere ayırır; bu tokenlar sayısal kimliklere çevrilir, embedding uzayına taşınır ve çok katmanlı attention bloklarından geçirilir. Eğitim sırasında amaç, verilen önceki tokenlara bakarak sıradaki token için olasılık dağılımı üretmektir. Basit görünen bu hedef, yeterli veri, model kapasitesi ve hesaplama ile dil bilgisi, kavramsal ilişkiler, kod yapıları, soru-cevap örüntüleri ve belirli ölçüde muhakeme davranışlarının birlikte öğrenilmesini sağlar.

Bir LLM'in kapasitesi yalnızca parametre sayısıyla açıklanamaz. Parametre sayısı modelin temsil gücünü etkilerken eğitim verisinin kalitesi, tokenizer tasarımı, context length, optimizasyon yöntemi, veri karışımı ve post-training süreci de sonuç üzerinde belirleyicidir. Örneğin 4 milyar parametreli iyi eğitilmiş bir model, belirli bir görevde daha büyük ancak zayıf veriyle eğitilmiş bir modeli geçebilir. Bu nedenle model seçerken yalnızca 'kaç milyar parametre' sorusu yerine görev türü, kullanılan dil, gecikme hedefi, GPU belleği, lisans, context length ve serving altyapısı birlikte değerlendirilmelidir.

LLM sistemleri iki ayrı katmanda düşünülmelidir: model katmanı ve uygulama katmanı. Model katmanı tokenizer, Transformer blokları, ağırlıklar, KV cache ve inference motorundan oluşur. Uygulama katmanında ise prompt yönetimi, RAG, tool calling, guardrail, caching, gözlemlenebilirlik ve kullanıcı oturumu gibi bileşenler bulunur. Kurumsal bir sistemde model tek başına ürün değildir; güvenilir bir LLM uygulaması modelin çevresindeki veri ve kontrol katmanlarıyla birlikte tasarlanır. Özellikle şirket içi dokümanlarla çalışan sistemlerde güncel bilgiye erişim, kaynak gösterme, yetkilendirme ve veri mahremiyeti model kalitesi kadar önemlidir.

Bu doküman teknik eğitim ve RAG uygulamalarında kullanılmak üzere hazırlanmıştır. İçerik uzun paragraflardan oluşur ve sonraki projede başlık bazlı değil, sabit karakter uzunluğu ve overlap kullanan character-based chunking yaklaşımıyla parçalanmaya uygundur. Böylece chunk sınırlarının anlamsal başlıklara tam oturmadığı durumlarda retrieval kalitesinin nasıl etkilendiği gözlemlenebilir.

2. Tokenization, Vocabulary ve Embedding

LLM'in gördüğü gerçek giriş metin değil token dizisidir. Tokenizer, UTF-8 metni modelin vocabulary'sinde bulunan alt parçalara dönüştürür. BPE, WordPiece ve SentencePiece benzeri algoritmalar sık görülen karakter ve kelime parçalarını tek token olarak tutarken nadir sözcükleri birden fazla alt parçaya ayırır. Bu nedenle token sayısı karakter veya kelime sayısıyla birebir aynı değildir. İngilizce için ortalama token/kelime oranı genellikle daha verimli iken Türkçe gibi eklemeli dillerde bir kelime birden fazla tokena bölünebilir. Context maliyeti, inference süresi ve API ücretlendirmesi gibi konular değerlendirilirken kelime değil gerçek tokenizer çıktısı dikkate alınmalıdır.

Vocabulary, tokenizer'ın tanıdığı token kümesidir. Vocabulary boyutu büyüdükçe bazı kelime ve kod parçaları daha az tokenla temsil edilebilir; ancak embedding ve çıktı katmanlarının boyutu da artar. Her token model içinde bir embedding vektörüne dönüştürülür. Bu vektör, tokenın yüksek boyutlu sayısal temsilidir. İlk embedding tablosu eğitim sırasında öğrenilir ve modelin sonraki katmanları bu temsilleri context'e göre sürekli dönüştürür. Aynı token farklı cümlelerde farklı hidden state'lere sahip olur; dolayısıyla LLM içindeki contextual representation ile bağımsız embedding modellerinin ürettiği retrieval embedding'leri aynı kavram değildir.

Pozisyon bilgisi de kritik bir bileşendir. Transformer'ın attention mekanizması token sırasını doğal olarak bilmez. Bu nedenle RoPE gibi positional encoding yöntemleri tokenların sıralı konumunu attention hesaplarına taşır. Uzun context modellerinde RoPE scaling, interpolation veya benzeri teknikler context uzunluğunu artırmak için kullanılabilir. Ancak yalnızca maksimum context değerinin yükseltilmesi, modelin uzak tokenlar arasındaki bilgiyi eşit kalitede kullanacağı anlamına gelmez. 'Lost in the middle' olarak bilinen davranışta model, çok uzun context içindeki orta bölümlerde yer alan bilgileri başlangıç ve sona göre daha zayıf kullanabilir.

RAG sistemlerinde tokenizer bilgisi chunk tasarımıyla doğrudan ilişkilidir. Character-based chunking 1000 karakterlik parçalar üretebilir fakat bu parçaların token sayısı değişebilir. Bir chunk'ın yaklaşık 1000 karakter olması, örneğin 250 token olacağı garantisini vermez. Production sistemlerinde character tabanlı yaklaşım kolay ve anlaşılır olduğu için kullanılabilir; daha kontrollü context yönetimi gerektiğinde tokenizer-aware chunking tercih edilebilir.

3. Transformer, Self-Attention ve Katman Yapısı

Transformer mimarisinin temel mekanizması self-attention'dır. Bir hidden state matrisi üç ayrı doğrusal dönüşümden geçirilerek Query, Key ve Value matrislerine dönüştürülür. Attention skoru temel olarak Query ile Key arasındaki benzerliği ölçer; skorlar ölçeklenip softmax ile normalize edilir ve Value vektörlerinin ağırlıklı toplamı alınır. Causal language model'lerde gelecekteki tokenlara bakılmasını engellemek için causal mask uygulanır. Böylece model her pozisyonda yalnızca kendisinden önce gelen tokenları kullanarak bir sonraki tokenı tahmin eder.

Multi-head attention, aynı token dizisini birden fazla attention head üzerinden işler. Her head farklı ilişki türlerine odaklanabilir; bazı head'ler yakın sözdizimsel ilişkileri, bazıları uzak referansları veya biçimsel kalıpları öğrenebilir. Attention çıktısından sonra feed-forward network katmanı gelir. Modern modellerde klasik ReLU yerine SiLU/SwiGLU gibi aktivasyon ve gating yapıları yaygındır. Her blokta residual connection ve normalization kullanımı, derin ağların stabil eğitimi için kritik önemdedir. RMSNorm, güncel decoder-only modellerde sık kullanılan normalization türlerinden biridir.

Model boyutu; katman sayısı, hidden dimension, attention head sayısı ve feed-forward dimension gibi hiperparametrelerle belirlenir. Parametrelerin önemli bölümü attention projeksiyonları ve feed-forward katmanlarında bulunur. Büyük modellerin inference maliyetini azaltmak için Multi-Query Attention (MQA) veya Grouped-Query Attention (GQA) gibi yöntemler kullanılabilir. GQA, çok sayıda query head'i daha az sayıda key/value head ile eşleştirerek KV cache boyutunu düşürür ve özellikle uzun context inference'ta önemli bellek avantajı sağlar.

Transformer katmanlarının ardışık çalışması sırasında her yeni token için önceki tokenlara ait key ve value tensörlerini tekrar hesaplamak verimsizdir. Bu nedenle autoregressive generation sırasında KV cache tutulur. KV cache inference performansının en kritik bileşenlerinden biridir ve uzun context, yüksek batch veya çoklu kullanıcı durumlarında GPU belleğinin büyük bölümünü tüketebilir. vLLM gibi inference motorlarının önemli optimizasyonlarından biri bu cache'i daha verimli yönetmektir.

4. Pretraining, Veri ve Optimizasyon

Pretraining aşamasında model çok büyük miktarda metin üzerinde next-token prediction ile eğitilir. Veri kaynakları web sayfaları, kitaplar, kod depoları, teknik dokümanlar, bilimsel metinler, forumlar ve lisanslı veri koleksiyonları gibi birçok kaynaktan gelebilir. Ham veri doğrudan modele verilmez; dil tespiti, kalite filtresi, deduplication, spam temizleme, kişisel veri filtreleme ve toksik içerik denetimi gibi işlemler uygulanır. Veri karışımı model davranışını doğrudan etkiler. Kod verisinin artırılması programlama yeteneğini, çok dilli verinin artırılması farklı dillerdeki performansı etkileyebilir.

Eğitim optimizasyonunda AdamW türevleri yaygın olarak kullanılır. Learning rate schedule genellikle warmup ile başlar ve cosine decay gibi bir yöntemle azaltılır. Büyük batch training, gradient accumulation ve distributed data parallel yaklaşımları çok sayıda GPU'nun birlikte kullanılmasını sağlar. Model çok büyüdüğünde tensor parallelism, pipeline parallelism, sequence parallelism ve optimizer state sharding gibi teknikler devreye girer. Eğitim sırasında mixed precision kullanmak hem bellek kullanımını hem de hesaplama hızını iyileştirir. BF16, geniş exponent aralığı nedeniyle modern GPU eğitimlerinde sık tercih edilir.

Pretraining maliyeti yalnızca GPU sayısıyla ölçülmez. Veri pipeline'ının GPU'ları sürekli besleyebilmesi, checkpoint altyapısı, ağ bant genişliği, storage throughput ve hata toleransı da toplam verimliliği etkiler. Model FLOPs utilization gibi metrikler donanımın teorik kapasitesinin ne kadarının gerçek eğitime dönüştüğünü anlamak için kullanılır. Büyük ölçekli eğitimde küçük bir iletişim veya veri okuma darboğazı bile toplam sürede büyük kayıplara neden olabilir.

Pretraining sonunda ortaya çıkan base model talimatları takip etmek zorunda değildir. Base model daha çok metin tamamlama davranışı gösterir. Kullanıcıya yardımcı olan chat modeli elde etmek için post-training gerekir. Bu ayrım önemlidir: aynı mimari ve parametre sayısına sahip iki modelden biri base, diğeri instruction-tuned olabilir ve kullanıcı deneyimleri tamamen farklı olabilir.

5. Instruction Tuning, Preference Alignment ve Reasoning

Instruction tuning, modelin kullanıcı talimatlarını daha iyi takip etmesi için soru-cevap, görev tanımı ve ideal yanıt çiftleri üzerinde supervised fine-tuning yapılmasıdır. Bu aşamada veri kalitesi miktardan daha kritik olabilir. İyi hazırlanmış örnekler modelin format takip etmesi, soruya doğrudan cevap vermesi, güvenli davranması ve belirli görevlerde beklenen çıktı yapısını öğrenmesini sağlar. Instruction tuning modelin pretraining sırasında öğrendiği bilgiyi yeniden öğretmekten çok, bu bilgiyi kullanıcı talebine göre nasıl kullanacağını şekillendirir.

Preference alignment aşamasında aynı prompt için birden fazla cevap karşılaştırılabilir. RLHF yaklaşımında insan tercihleriyle bir reward model eğitilir ve politika reinforcement learning ile optimize edilir. DPO gibi yöntemler ise tercih çiftlerini daha doğrudan kullanarak eğitim sürecini sadeleştirir. Güncel model geliştirme süreçlerinde insan verisine ek olarak model tarafından üretilen sentetik tercih verileri ve verifier tabanlı ödüller de kullanılabilir. Amaç her zaman 'daha uzun cevap' üretmek değildir; doğruluk, yardımseverlik, güvenlik, format ve görev başarısı gibi çok boyutlu hedefler dengelenir.

Reasoning modellerinde modelin cevaptan önce daha fazla ara hesaplama yapmasına izin veren training ve inference teknikleri kullanılabilir. Test-time compute arttıkça bazı matematik, kodlama ve planlama görevlerinde başarı artabilir. Ancak her soru reasoning gerektirmez. Basit RAG soru-cevap sisteminde uzun düşünme çıktıları latency ve token tüketimini artırabilir. Bu nedenle üretim sistemlerinde task routing ile basit sorgular hızlı modele, karmaşık sorgular reasoning moduna yönlendirilebilir.

Reasoning çıktısının her zaman doğru olduğu varsayılmamalıdır. Model hatalı bir varsayımdan başlayıp tutarlı görünen uzun bir açıklama üretebilir. Bu nedenle kurumsal RAG sistemlerinde reasoning'i kaynak retrieval, doğrulama, structured output ve gerektiğinde tool use ile desteklemek önemlidir. Modelin düşünme kapasitesi, dış dünyadaki doğru bilgiye erişim ihtiyacını ortadan kaldırmaz.

6. Inference, Sampling ve KV Cache

Inference sırasında model prompt tokenlarını işler ve ardından autoregressive olarak yeni token üretir. İlk aşama prefill, sonraki aşama decode olarak düşünülebilir. Prefill sırasında prompt içindeki tokenlar paralel işlenebilir; decode sırasında ise her yeni token önceki çıktıya bağlı olduğu için adımlar sıralı ilerler. Bu nedenle uzun promptlar time-to-first-token değerini, uzun cevaplar ise toplam generation latency'yi artırır. Serving sistemlerinde throughput ve latency arasında doğrudan trade-off bulunur.

Modelin logits çıktısı sampling parametreleriyle token seçimine dönüştürülür. Temperature logits dağılımını keskinleştirir veya düzleştirir. Düşük temperature daha deterministik, yüksek temperature daha çeşitli sonuç üretir. Top-p nucleus sampling, toplam olasılığı belirli bir eşiğe ulaşan en olası token kümesinden seçim yapar. Top-k ise yalnızca en olası k tokenı aday bırakır. RAG ve kurumsal soru-cevap senaryolarında düşük temperature genellikle daha uygunken yaratıcı üretimde daha yüksek değerler kullanılabilir.

KV cache, daha önce işlenen tokenların key/value tensörlerini saklayarak decode sırasında tekrar hesaplamayı önler. Cache boyutu model katman sayısı, KV head sayısı, head dimension, sequence length, batch size ve veri tipine bağlıdır. Uzun context ve çok sayıda eşzamanlı kullanıcı GPU belleğini hızla tüketebilir. Bu nedenle serving kapasitesi yalnızca model ağırlıklarının VRAM'e sığıp sığmadığıyla belirlenemez; KV cache için de yeterli alan bırakılmalıdır.

Continuous batching, farklı kullanıcı isteklerini decode sırasında dinamik biçimde aynı batch içine alarak GPU kullanımını artırır. vLLM, PagedAttention yaklaşımı ile KV cache bloklarını sayfalı bellek mantığına benzer şekilde yöneterek parçalanmayı azaltır. Bu özellikler on-prem LLM servisinde yüksek eşzamanlılık için önemlidir. Tek kullanıcı testinde hızlı görünen bir model, gerçek üretimde batching ve cache yönetimi zayıfsa düşük throughput verebilir.

7. Quantization, Donanım ve vLLM Serving

LLM ağırlıkları FP32 yerine FP16 veya BF16 formatında tutulduğunda bellek ihtiyacı yaklaşık yarıya iner. Quantization ile ağırlıklar INT8, INT4 veya özel düşük bit formatlarına indirilebilir. 4-bit quantization modelin GPU belleği ihtiyacını ciddi ölçüde azaltabilir; ancak kalite, kernel desteği ve hız her yöntemde aynı değildir. AWQ ve GPTQ gibi weight-only yöntemleri inference için yaygın seçeneklerdir. Quantization seçerken yalnızca model dosyasının küçülmesine değil kullanılan inference motorunun ilgili kernel için optimize olup olmadığına da bakılmalıdır.

GPU seçiminde VRAM, memory bandwidth ve tensor core yetenekleri önemlidir. Küçük bir 4B model tek güçlü tüketici GPU'sunda rahat çalışabilirken daha büyük modeller tensor parallelism ile birden fazla GPU'ya dağıtılabilir. Tensor parallelism, ağırlık matrislerini GPU'lar arasında bölerek aynı katmanın paralel hesaplanmasını sağlar. Bunun karşılığında GPU'lar arası hızlı iletişim gerekir. PCIe üzerinden dağıtılan model ile NVLink/NVSwitch altyapısındaki model aynı ölçeklenmeyi göstermeyebilir.

vLLM, Hugging Face modelini OpenAI-compatible HTTP API şeklinde sunabilir. Uygulama katmanı doğrudan model nesnesine bağlanmak yerine HTTP üzerinden `/v1/chat/completions` benzeri endpoint kullanır. Böylece model servisi ile RAG uygulaması birbirinden ayrılır. Model yeniden başlatılabilir, farklı GPU'ya taşınabilir veya aynı API sözleşmesi korunarak başka bir modele geçilebilir. Bu ayrım kurumsal mimaride deployment, ölçekleme ve gözlemlenebilirliği kolaylaştırır.

Serving testinde tokens per second tek başına yeterli değildir. Time to first token, inter-token latency, p50/p95 latency, concurrent request sayısı, GPU utilization ve out-of-memory oranı birlikte izlenmelidir. Ayrıca context length arttıkça aynı donanım üzerindeki kullanıcı kapasitesi düşebilir. Bu nedenle benchmark gerçek uygulamadaki prompt uzunluğu ve cevap uzunluğuna yakın yüklerle yapılmalıdır.

8. RAG, Embedding ve Vector Search

Retrieval Augmented Generation, LLM'in parametrelerinde bulunmayan veya güncel olmayan bilgiyi dış veri kaynağından getirerek prompt context'ine ekler. Temel RAG akışında dokümanlar parçalanır, her parça embedding modelinden geçirilir ve vector database'e kaydedilir. Kullanıcı sorusu aynı embedding uzayına dönüştürülür, en yakın vectorler bulunur ve ilgili metin parçaları LLM'e verilir. Böylece model cevabı kurumsal kaynaklara dayandırabilir.

Embedding modeli ile generative LLM farklı görevler yapar. Embedding modeli metni sabit boyutlu bir vector'e dönüştürür; amacı benzer anlamdaki metinleri uzayda yakın konumlandırmaktır. Generative LLM ise token dizisi üretir. Retrieval kalitesi kötü olduğunda çok güçlü bir LLM bile doğru dokümanı görmediği için güvenilir cevap veremez. Bu nedenle embedding modeli, chunking, retrieval parametreleri ve reranking RAG kalitesinin temel parçalarıdır.

Vector search için cosine similarity, inner product veya L2 distance kullanılabilir. Normalize edilmiş embedding'lerde inner product cosine similarity ile aynı sıralamayı verebilir. FAISS gibi kütüphaneler local similarity search için yeterliyken Milvus gibi vector database'ler collection, metadata, persistence, filtering, hybrid search ve ölçekleme gibi ek özellikler sunar. Büyük kurumsal sistemlerde vector database seçimi sadece arama hızına değil operasyon, yedekleme, erişim kontrolü ve veri yaşam döngüsüne göre yapılmalıdır.

Character-based chunking, metni sabit karakter uzunluğunda parçalara ayırır. Örneğin chunk_size=1000 ve chunk_overlap=200 seçildiğinde ilk parça 0-999, sonraki parça yaklaşık 800-1799 karakterlerini kapsar. Overlap, chunk sınırında bölünen bir bilginin komşu parçada tekrar bulunmasını sağlar. Bunun dezavantajı semantik bölüm sınırlarını dikkate almamasıdır. Bu doküman sonraki projede tam olarak bu trade-off'u gözlemlemek için kullanılabilir.

9. Hybrid Search, Reranking ve Query Rewriting

Dense semantic search anlam benzerliğinde güçlüdür ancak belirli ürün kodları, hata numaraları, kısaltmalar veya nadir teknik terimlerde lexical search daha iyi sonuç verebilir. BM25 kelime eşleşmesi ve term frequency mantığıyla çalışır. Hybrid search, dense vector search ile BM25 sonucunu birleştirir. Reciprocal Rank Fusion (RRF) gibi yöntemler iki farklı sıralamadaki sonuçları tek bir liste haline getirir. Milvus gibi sistemler dense ve sparse retrieval'ı aynı collection üzerinde yöneterek bu mimariyi basitleştirebilir.

Reranking, ilk retrieval aşamasında getirilen daha geniş aday listesini daha güçlü fakat daha pahalı bir modelle yeniden sıralamaktır. Bi-encoder embedding yaklaşımında query ve passage ayrı ayrı vector'e çevrilir ve çok hızlı karşılaştırılır. Cross-encoder reranker ise query ile passage'ı aynı anda modele vererek doğrudan relevance skoru üretir. Bu nedenle binlerce doküman üzerinde ilk arama için pahalıdır fakat Top-20 veya Top-50 adayın yeniden sıralanmasında çok etkilidir. BGE reranker ailesi bu amaçla kullanılabilecek çok dilli seçeneklerden biridir.

Query rewriting, kullanıcının ham sorusunu retrieval için daha açık ve bağımsız bir sorguya dönüştürür. Özellikle conversational RAG'de kullanıcı 'peki raporu nereye vereceğim?' gibi önceki konuşmaya bağlı sorular sorabilir. Bir LLM, son konuşma geçmişini kullanarak bu ifadeyi 'Hastalık izni için sağlık raporu hangi birime iletilmelidir?' şeklinde yeniden yazabilir. Retrieval bu standalone query üzerinde yapılırken final cevap kullanıcıya orijinal konuşma bağlamında verilir.

Query rewriting'in kontrolsüz kullanımı sorgunun anlamını değiştirebilir. Bu nedenle production sisteminde original query ve rewritten query birlikte loglanmalı, gerekirse rewrite yalnızca follow-up sorularda uygulanmalı ve evaluation set üzerinde retrieval başarısına etkisi ölçülmelidir. Rewrite sonrası query'nin çok genişlemesi veya kullanıcı niyetinden uzaklaşması yanlış chunk'ların getirilmesine neden olabilir.

10. Conversational RAG, Evaluation ve Production Güvenilirliği

Conversational RAG sistemi yalnızca geçmiş mesajları prompt'a eklemekten ibaret değildir. Retrieval için kullanıcının mevcut niyetini doğru temsil eden bağımsız bir sorgu üretmek, generation için ise gerekli konuşma bağlamını korumak gerekir. Uzun konuşmalarda bütün geçmişi her istekte taşımak context maliyetini yükseltir. Son N mesaj, kısa conversation summary ve kullanıcıya ait kalıcı olmayan session state birlikte kullanılabilir. Önemli olan retrieval query ile final generation context'inin farklı ihtiyaçları olduğunu ayırmaktır.

RAG evaluation iki katmanda yapılabilir: retrieval evaluation ve answer evaluation. Retrieval tarafında Hit@K, Recall@K ve Mean Reciprocal Rank gibi metrikler doğru passage'ın sonuç listesine girip girmediğini ölçer. Örneğin 20 soruluk test setinde doğru chunk 17 soruda ilk üç sonuç içinde bulunuyorsa Hit@3 yüzde 85'tir. MRR ise doğru sonucun sıralamadaki konumuna duyarlıdır; doğru passage birinci sıradaysa 1, ikinci sıradaysa 0.5, üçüncü sıradaysa yaklaşık 0.33 katkı yapar.

Answer evaluation tarafında cevap doğruluğu, faithfulness, kaynak kullanımı ve completeness değerlendirilebilir. LLM-as-a-Judge otomasyon sağlayabilir ancak judge modelinin de bias ve hata riski vardır. Kritik sistemlerde örneklem bazlı insan değerlendirmesi ve otomatik metriklerin birlikte kullanılması daha güvenlidir. En değerli yaklaşım, yeni bir retrieval tekniği eklendiğinde aynı golden test seti üzerinde önce/sonra karşılaştırması yapmaktır. Böylece query rewrite, hybrid search veya reranker'ın gerçekten fayda sağlayıp sağlamadığı ölçülebilir.

Production sisteminde gözlemlenebilirlik zorunludur. Loglarda request ID, model adı, prompt versiyonu, retrieval sonuçları, similarity/reranker skorları, latency, token kullanımı ve hata bilgileri bulunabilir. Hassas kullanıcı verileri ham biçimde loglanmamalıdır. Ayrıca prompt injection, yetkisiz doküman erişimi ve cross-tenant data leakage gibi RAG'e özgü güvenlik riskleri dikkate alınmalıdır. Retrieval katmanı kullanıcı yetkisini ihlal ediyorsa LLM'in güvenlik prompt'u bu problemi çözemez.

Sonuç olarak güvenilir bir on-prem RAG sistemi; iyi bir embedding modeli, doğru chunking, vector database, hybrid retrieval, reranking, conversation-aware query rewriting, güçlü fakat kontrollü bir LLM ve ölçülebilir evaluation sürecinin birlikte çalışmasıyla oluşur. Model yalnızca zincirin bir parçasıdır. Gerçek üretim kalitesi, verinin hazırlanmasından serving altyapısına ve izleme mekanizmalarına kadar bütün pipeline'ın birlikte tasarlanmasına bağlıdır.